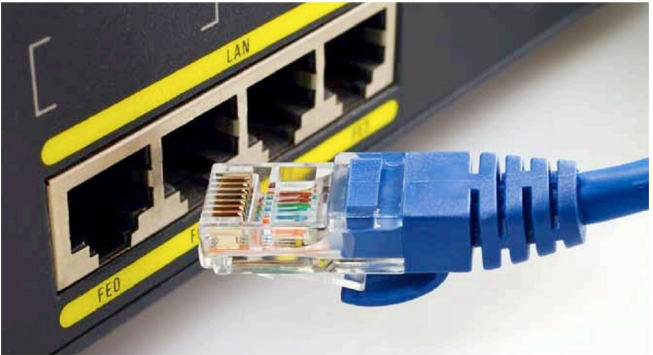


An Introduction to Socket Programming in C (UDP)

This article explores the basics of writing programs using sockets in C, with BSD sockets and the IP protocol version 4.



Network applications, like file transfer programs, need to conform to some communication protocol. The most common protocol is TCP/IP, which is layered—for example, application, transport, network and link layers.

You can actually look at a socket as an extension of inter-process communication techniques like pipes, FIFOs, message queues, etc. These are limited to a single machine. There are also UNIX Domain Sockets, which can be used to communicate only between processes on the same machine. However, I am talking about TCP, UDP or SCTP inter-machine network sockets that use the services of the transport layer and can help to directly get the services of the network layer (raw sockets). With these sockets, in addition to local communication on a single machine, you can communicate between different machines running different operating systems—and each program can be written in a different programming language from the other. Here, we look at the

basics of using UDP sockets.

What does a socket program look like? A typical socket program might have two source files—one for the *client* and one for the *server*. “For instance, my source files are *lfy_udp_client.c* and *lfy_udp_server.c*, which can be downloaded from <https://github.com/mybodhizone/udp-socket>.” The two code snippets below represent two terminal windows, with each program compiled and the executables run—and the output is shown below:

```
[localhost udp]$ gcc lfy_udp_server.c -o myserver
[localhost udp]$ ./myserver
UDP Server - Waiting for Client data
Data Received from 127.0.0.1
I have received Testing

UDP Server - Waiting for Client data
```